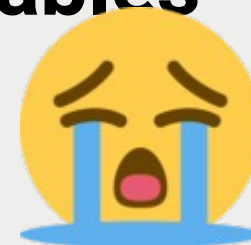


Testing

Introducción

Contexto y Motivación

- El mito dice si nos concentramos y programamos bien **no vamos a tener errores**. Entonces si somos buenos programadores no deberían tener.
- Si hay **bugs**, es porque somos **malos programadores**,
- Si somos malos debemos sentirnos **culpables**



Contexto y Motivación

- El mito dice si nos concentramos y programamos bien **no vamos a tener errores**. Entonces si somos buenos programadores no deberían tener.
- Si hay bugs, es porque somos malos programadores
- Si somos malos debemos sentirnos culpables

FALSO!



La **producción de software**, como cualquier actividad humana, es un proceso **sujeto a errores**. Cualesquiera sean los métodos usados, **no** es posible **garantizar** que el **producto final** se comporte siempre **como es requerido**

Definición

- Esta idea general también es apropiada para el desarrollo de software, llevándonos a la **noción de Testing de Software**

Cem Kaner ✓

*Professor of the Florida Institute of Technology
Director Center for Software Testing Education & Research
Founder Association for Software Testing*

*El **testing de software** es una actividad de investigación realizada para proveer a los stakeholders información sobre la calidad del producto de software o servicio que se testea*

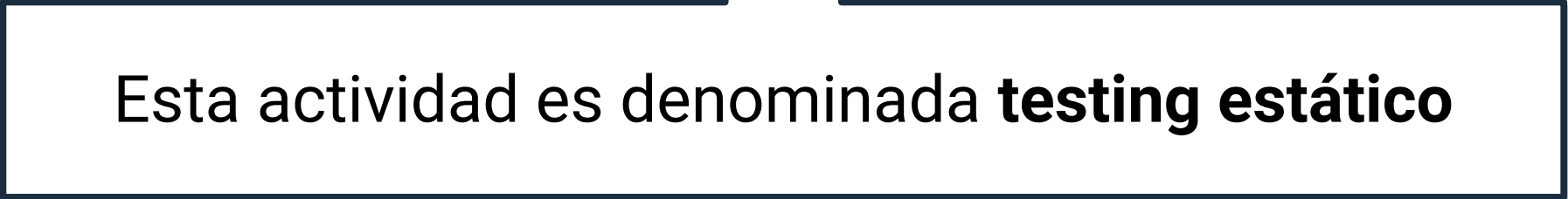


La Primer Clasificación

- La única forma de **examinar** directamente el software es **mirando su código** y su documentación
 - El **comportamiento dinámico** del software es otro problema. Observando el código o la documentación no podemos evaluar este comportamiento
- El **comportamiento dinámico** es evaluado mediante la **ejecución** del software
 - Durante esta **ejecución**, el comportamiento del software se **compara** con el comportamiento esperado.

La Primer Clasificación

- La única forma de **examinar** directamente el software es **mirando su código** y su documentación



– Esta actividad es denominada **testing estático**

no podemos evaluar este comportamiento

- El **comportamiento dinámico** es evaluado mediante la **ejecución** del software



– Esta actividad es denominada **testing dinámico**

esperado.

La Primer Clasificación

- **Testing Estático:**
 - **No se lo suele llamar “testing”** sino que directamente se usa el nombre de la actividad que estamos haciendo
 - **Actividades** más comunes
 - **Code Review y Proofreading**

Por ejemplo, si estamos revisando el código de un compañero para ver si tiene smells, decimos que estamos haciendo Code Review, no decimos que estamos haciendo testing estático

La Primer Clasificación

- **Testing Dinámico:**

- Es lo que normalmente se conoce como “**testing**”
 - Es el testing que implica ejecutar lo que queremos testear
- En lo que sigue de la materia cuando hablemos de **testing** nos vamos a referir al testing dinámico

Si bien vamos a **clasificar aún más** al testing, a continuación vamos a ver varias **nociones básicas** que son **comunes** a todas las actividades

Nociones Básicas

Nociones Básicas de Testing

- Al elemento que estamos testeando lo llamamos **objeto del test**
- La **ejecución** de un objeto de test se realiza determinando condiciones de ejecución, la entrada y la salida esperada, denominados **datos de test**

Un **caso de test** se caracteriza por:

- El objeto de test (qué quiero probar)
- Los datos de test (cómo quiero probarlo y qué espero)

Nociones Básicas de Testing

- Al **probar** una función, lo llamamos **caso de test**
 - La **función** que se realiza es la **función**, la **función** que se realiza con **datos** de test
- Por ejemplo, asumiendo que tenemos una función que calcula los divisores de un número, un **caso de test** podría ser **probar divisores(X) con X=6 y el resultado esperado es [2,3]**

Un **caso de test** se caracteriza por:

- El objeto de test (qué quiero probar)
- Los datos de test (cómo quiero probarlo y qué espero)

Nociones Básicas de Testing

- Los casos de test buscan ser “destructivos”:
 - Son útiles si se encuentran fallas no descubiertas
- Cuando ejecutamos un caso de prueba podemos tener, en general, **dos resultados**
 - **Éxito**: Se obtiene el resultado esperado
 - **Falla**: Cuando el resultado obtenido por el objeto de prueba es diferente al resultado esperado

Nociones Básicas de Testing

- Los casos de test buscan ser “destructivos”:
 - Son útiles si se encuentran fallas no descubiertas.
- Cuando ejecutamos un caso de prueba podemos tener, en general, **dos resultados**
 - **Éxito**: Se obtiene el resultado esperado
 - **Falla**: Cuando el resultado obtenido por el objeto de prueba es diferente al resultado esperado

El caso de prueba nos sirvió para detectar que hay un **defecto el sistema**

Nociones Básicas de Testing

- Los casos de test buscan ser “destructivos”:
 - Son útiles si se encuentran fallas no descubiertas.
- Cuando ejecutamos un caso de prueba podemos tener, en general, **dos resultados**
 - **Éxito**: Se obtiene el resultado esperado
 - **Falla**: Cuando el resultado obtenido por el objeto de prueba es diferente al resultado esperado



El caso de prueba nos sirvió para detectar que hay un **defecto el sistema**

Sin embargo, no nos dice dónde está ese defecto ni cómo arreglarlo!

Es es el “**debugging**”

Testing

Testing de Unidad - Conceptos

Contexto

- En esta presentación vamos a profundizar los conceptos vinculados al Testing de Unidad
- Antes de arrancar aclaremos que algunos términos son sinónimos
 - *Testing de Unidad, Testeo de Unidad, Testing Unitario, Testeo Unitario*: es la actividad armar tests de este tipo, correrlos, estudiar sus resultados
 - *test unitario, test de unidad, prueba unitaria, prueba de unidad*: refiera a un caso de prueba/test del Testing de Unidad

¿Qué era el Testing de Unidad?

- Es el tipo de test de más bajo nivel y se hace directamente sobre código.
- Sus tests prueban las unidades del código
 - Una porción “unitaria” del código

¿Qué es esa “unidad” o “porción unitaria”?

Qué es la unidad depende del equipo/proyecto. Lo más común suele ser un método, una función o una clase, pero también un grupito de clases

El equipo decide el “tamaño” de la unidad y si va a haber algunas excepciones

Contexto - ¿Qué no es un test unitario?

- Los test de unidad **sólo prueban correctitud**. No prueban, por ejemplo, performance, stress o usabilidad
- En general, los tests unitarios **no prueban los sistemas** que están **fuera** de la base código
 - Por ejemplo, no testean que ande la base de datos, que esté bien cargada, o que un servicio web externo esté andando bien

Anatomía de un Test en el Testing Unitario

- Básicamente, un caso de test de en el testing de unidad es una porción de código que:
 - **Arrange**: **Setea la unidad** para que esté en un estado donde pueda ejecutarse de forma aislada (crear objetos, establecer dependencias, etc.)
 - Esto puede implicar determinar los **datos de entrada** del test
 - **Act**: **Ejecuta la unidad** (o lo que haya que ejecutar de la unidad si es algo más grande que una función)
 - **Assert**: **Contrasta el resultado** de la ejecución con el **resultado esperado** del test

Ejemplo a mano de un Test Unitario

- Veamos un ejemplo simple de cómo hacer **un** test de unidad hecho **EXTREMADAMENTE a mano**
 - Supongamos que queremos probar el método **sumar** de esta

```
class Calculador {  
    fun sumar(x: Int, y: Int): Int {  
        return x + y  
    }  
}
```

- Por simplicidad y para ejemplificar, podríamos implementar el test de unidad directamente en el main

Ejemplo a mano de un Test Unitario

- Veamos un ejemplo simple de cómo hacer **un** test de unidad hecho **EXTREMADAMENTE a mano**
 - Supongamos que queremos probar el método **sumar** de esta

```
class Calculador {  
    fun sumar(x: Int, y: Int): Int {  
        return x + y  
    }  
}
```

- Por simplicidad y para ejemplificar, podríamos implementar el test de unidad directamente en el main

```
fun main() {  
    val calc = Calculador()  
    val v1 = 2; val v2 = 5; val esperado = 7
```

Arrange

)

Ejemplo a mano de un Test Unitario

- Veamos un ejemplo simple de cómo hacer **un** test de unidad hecho **EXTREMADAMENTE a mano**
 - Supongamos que queremos probar el método **sumar** de esta

```
class Calculador {  
    fun sumar(x: Int, y: Int): Int {  
        return x + y  
    }  
}
```

- Por simplicidad y para ejemplificar, podríamos implementar el test de unidad directamente en el main

```
fun main() {  
    val calc = Calculador()  
    val v1 = 2; val v2 = 5; val esperado = 7
```

Arrange

Act

```
    val resultado = calc.sumar(v1, v2)
```

")

Ejemplo a mano de un Test Unitario

- Veamos un ejemplo simple de cómo hacer **un** test de unidad hecho **EXTREMADAMENTE a mano**
 - Supongamos que queremos probar el método **sumar** de esta

```
class Calculador {  
    fun sumar(x: Int, y: Int): Int {  
        return x + y  
    }  
}
```

- Por simplicidad y para ejemplificar, podríamos implementar el test de unidad directamente en el main

```
fun main() {  
    val calc = Calculador()  
    val v1 = 2; val v2 = 5; val esperado = 7  
  
    val resultado = calc.sumar(v1, v2)  
  
    if (resultado != 7) {  
        throw IllegalStateException("El resultado de la sumar $v1 con $v2 fue $resultado, pero se esperaba $esperado.")  
    } else {  
        println("Test de sumar($v1, $v2) ejecutado manualmente: ¡Correcto!")  
    }  
}
```

Arrange

Act

Assert

Otras Características: Independencia

- Los **tests** unitarios deben ser **independientes entre sí**
- Es decir, la ejecución de un test **no debería afectar** la ejecución de otro
 - Si vamos a correr varios tests distintos sobre un objeto que puede cambiar de estado, en el seteo de la unidad deberíamos tener esto en cuenta
 - Es parte del aislamiento de la unidad
 - Es por esto que trabajamos con conjuntos de tests y no con secuencias

Otras Características: Probar una sola cosa

- Cada test unitario debe probar una **única responsabilidad** a la vez
- Podemos probar **más de una cosa**, pero siempre debe ser sobre la misma responsabilidad
 - En particular, para los **test negativos** sí es importante probar **una sola cosa**
 - Al ser negativo vamos a probar situaciones a donde la unidad va a fallar...

Si la unidad falla, no podemos seguir probando cosas sobre ella sin volver a hacer su seteo... lo cual termina siendo un nuevo test!

Otras Características: Dependencias

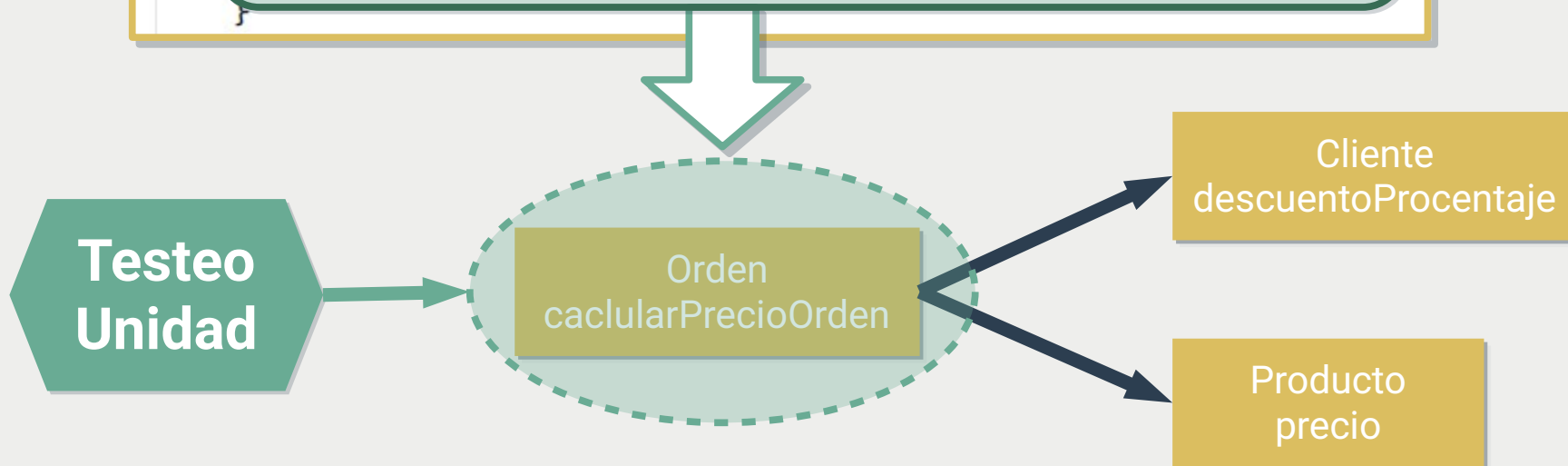
- Es común que una **unidad** que queremos probar **dependa** de otras **unidades**

```
class Orden(  
    val cliente: Cliente,  
    val producto: Producto,  
    val cantidadProducto: Int  
) {  
    fun calcularPrecioOrden(): Float {  
        val valorNominal = producto.precio() * cantidadProducto  
        val descuento = valorNominal * cliente.descuentoPorcentaje()  
        return valorNominal - descuento  
    }  
}
```

Otras Características: Dependencias

- Es común que una **unidad** que queremos probar **dependa** de otras **unidades**

```
class Orden(  
    val cliente: Cliente,  
    val producto: Producto,  
    val cantidadProducto: Int  
) {  
    fun calcularPrecioOrden(): Float {  
        val valorNominal = producto.precio() * cantidadProducto  
        val descuento = valorNominal * cliente.descuentoPorcentaje()  
        return valorNominal - descuento  
    }  
}
```



Otras Características: Dependencias

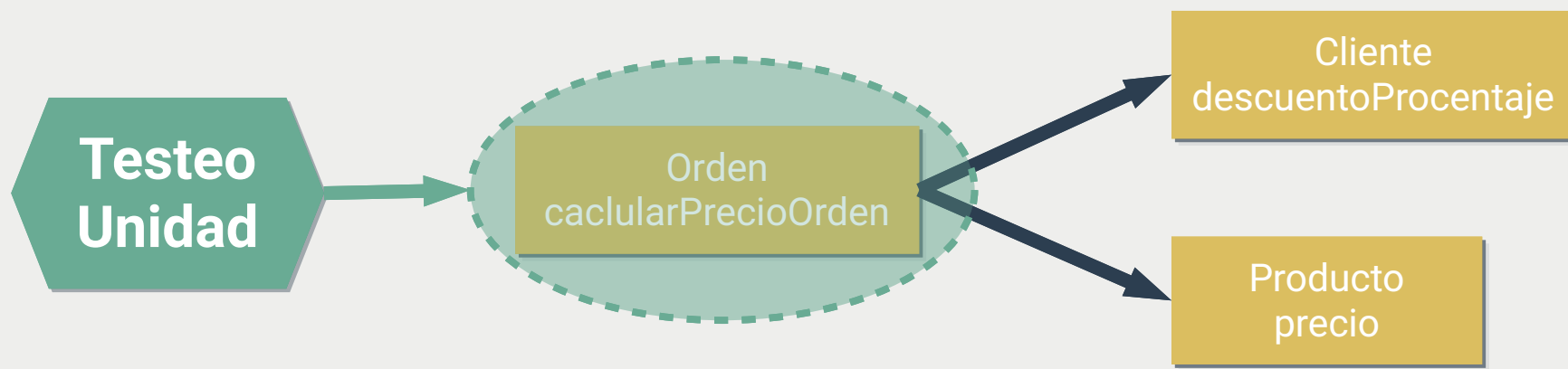
- Para manejar una dependencia en el testing de unidad tenemos dos alternativas:

Testeo Sociable

Testeo Solitario

Otras Características: Testeo Sociable

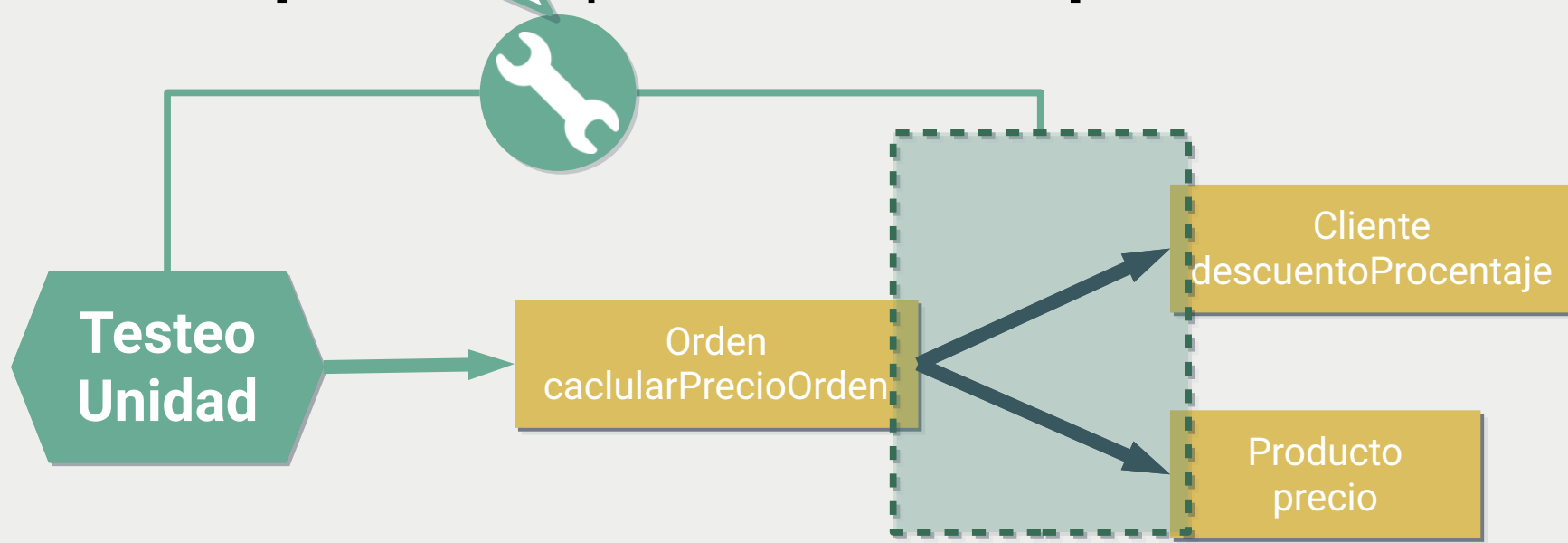
- En el testeo sociable mantenemos las dependencias reales del objeto de prueba
- La idea es mantener las dependencias reales siempre que sea posible/razonable
 - Si dependemos de algo **inestable o impredecible**, lo **reemplazamos** por un **doble de prueba**



Otras Características: Testeo Sociable

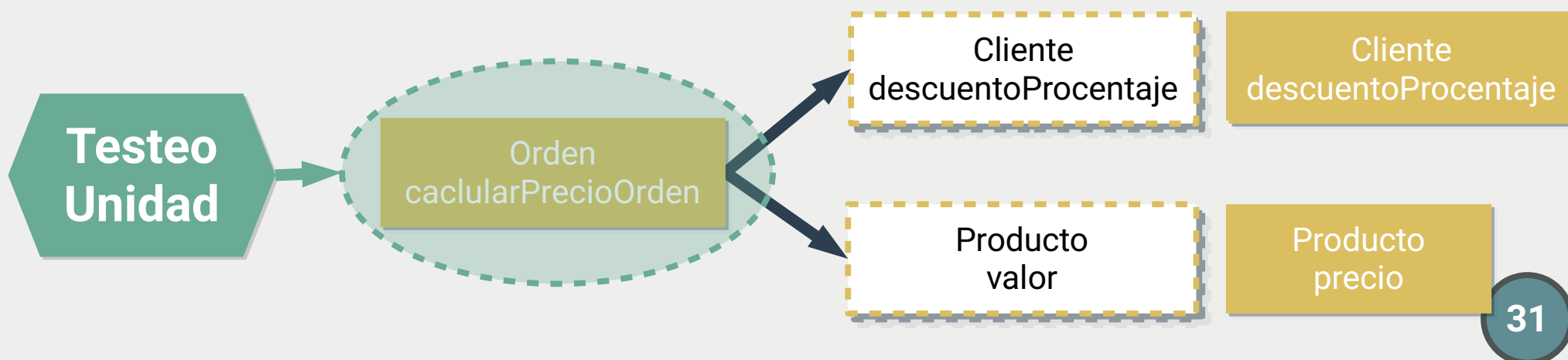
- En el testeo sociable mantenemos las dependencias reales del objeto de prueba
- La idea es que el test unitario sea el que tenga dependencias reales razonables
 - Si dependencias inestable o impredecible, lo reemplazamos por un doble de prueba

Es posible que el test unitario sea el que tenga dependencias reales que asociar la unidad con sus dependencias reales



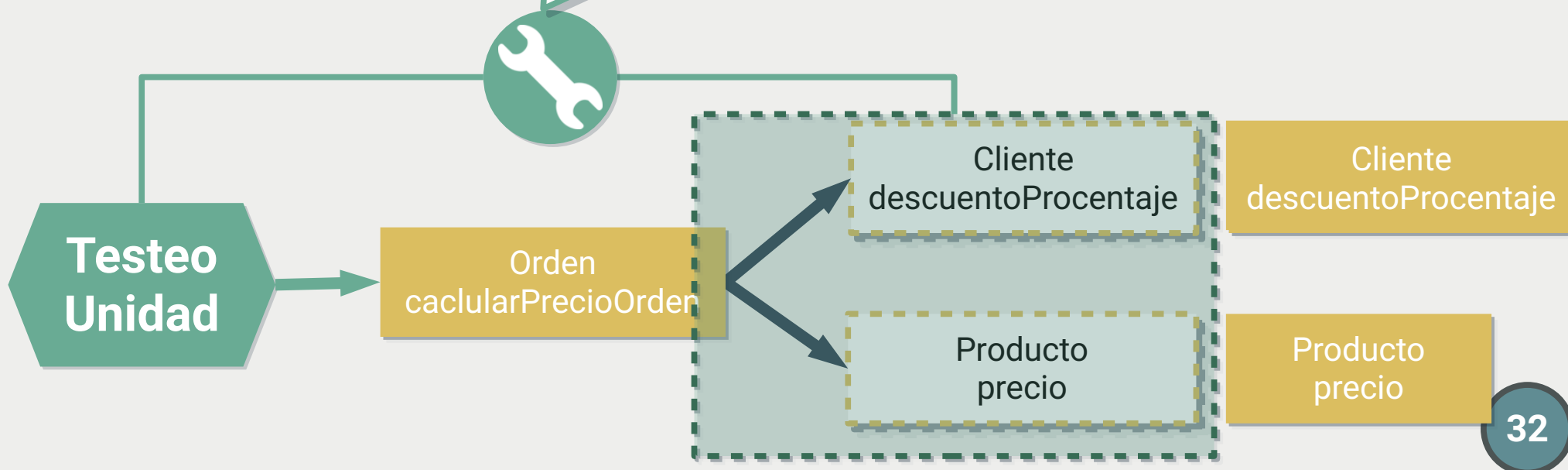
Otras Características: Testeo Solitario

- En el testeo solitario no queremos usar la dependencia real porque cualquier falla en ella podría hacernos fallar la unidad que estamos probando
 - Para eso usamos un “**Doble de Prueba**” (test doble) en vez del elemento real del que dependemos



Otras Características: Testeo Solitario

- En el testeo solitario no queremos usar la
cualquier falla en ella
unidad que estamos
El test unitario es el encargado
de crear los dobles y
establecer las dependencias
con el objeto de prueba
en vez del elemento real del que dependemos



Otras Características: Testeo Solitario

- En el testeo solitario no queremos usar la dependencia real porque cualquier falla en ella podría hacernos fallar la unidad que estamos probando
 - Para eso usamos un “**Doble de Prueba**” (test doble) en vez del elemento real del que dependemos

Esta política de manejo de dependencias busca favorecer el **aislamiento** de las unidades que estamos **testeando**

Definición y Conceptos

Un **doble de prueba** es una **versión alternativa** de un elemento del código cuyo comportamiento es **simplificado o alterado** para que **beneficie** a los objetivos del **testing** cuando **reemplaza** al elemento original

- Los elementos pueden ser objetos, clases, funciones, métodos
 - Cosas de las que depende la unidad que vamos a testear

Doble - Cliente
descuentoProcentaje

Cliente
descuentoProcentaje

Definición y Conceptos

- La intuición es que vamos a **reemplazar** al elemento **original** por su **doble** en un test
- El **doble** debería **verse y responder** a las mismas cosas que el original (e.g. tener los mismos métodos)
- Pero va a responder utilizando **respuestas definidas a mano**, especificadas dentro del test o antes

Doble - Cliente

...
descuentoProcentaje()

return 0.1;

Cliente

...
descuentoProcentaje()

```
if(esClienteVip){  
    return añosActividad()*  
           formula()/100;  
}else{  
    ...  
}
```

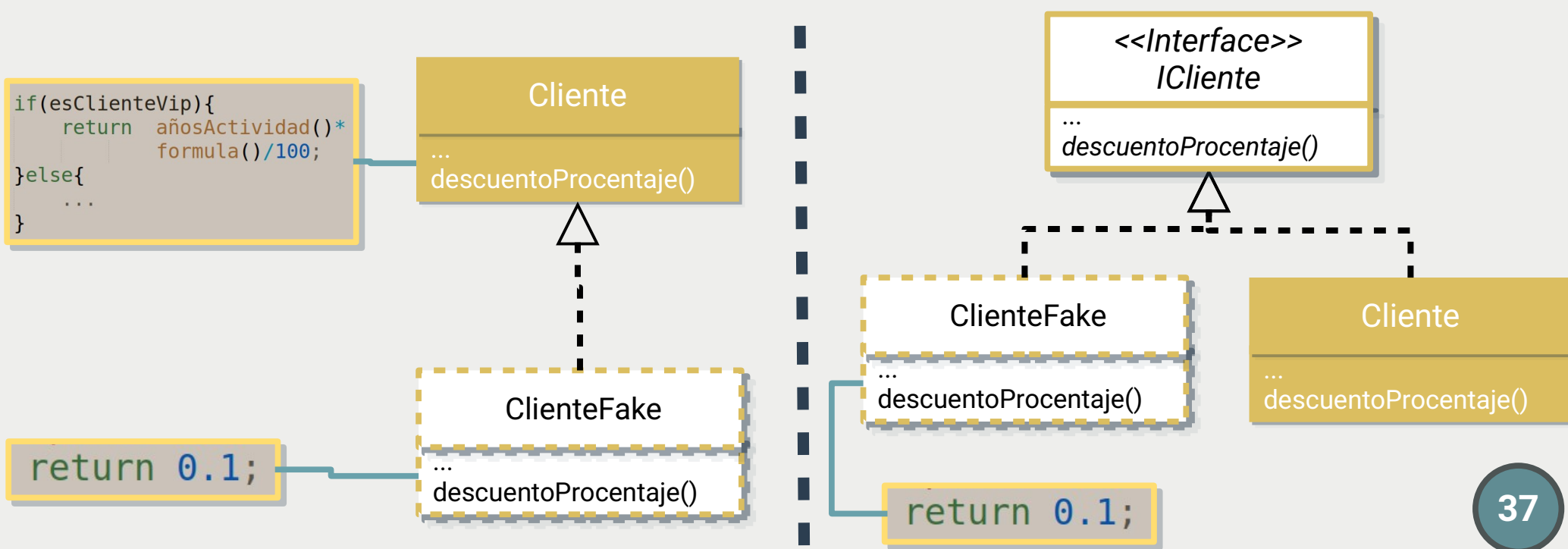
Definición y Conceptos – Mocks y Fakes

- Hay varios tipos de dobles de prueba
- Los más comunes son los
 - **Mock Objects**
 - **Test Fakes** (también llamados stubs)
- Conceptualmente son muy parecidos

La diferencia principal es en qué momento se determina el comportamiento del doble. En los mocks se hace dentro de cada test, mientras que en los fakes se hace de forma general

Definición y Conceptos – Test Fakes

- Entonces es común que armemos los **Fakes** utilizando las herramientas de polimorfismo del lenguaje
 - Podemos **heredar** del objeto que queremos imitar o de su interface (si la tiene) y **redefinir/implementar** los métodos necesarios



Definición y Conceptos – Test Fakes

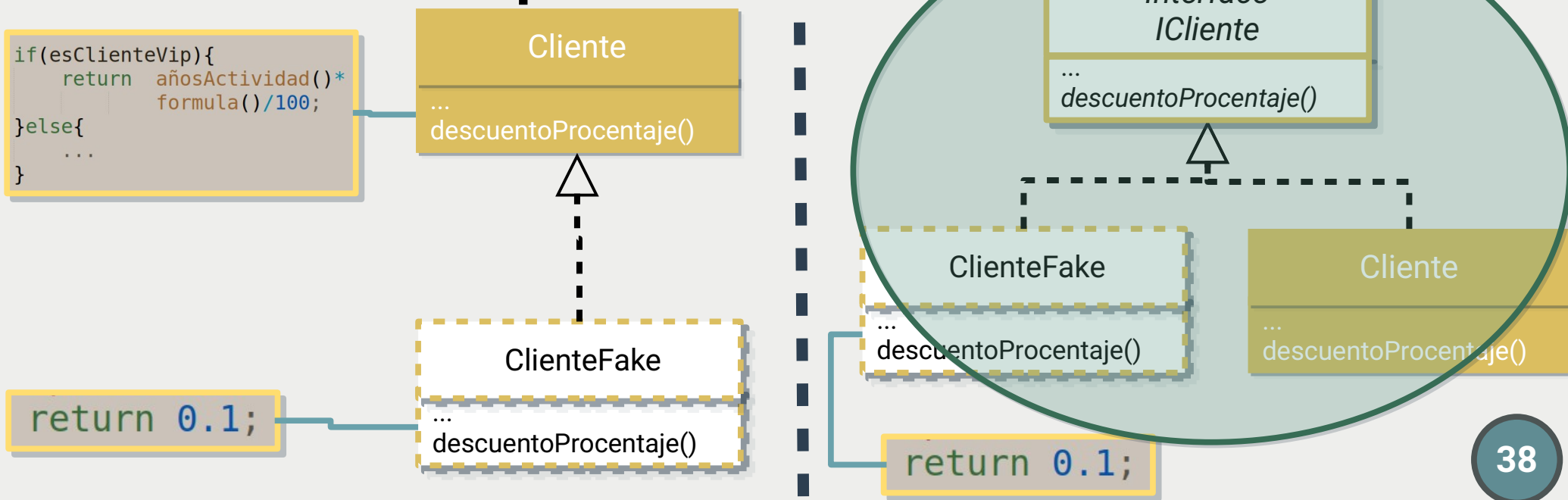
- Entonces **hacemos los Fakes**

utilizando
lenguaje

Esta es la **forma recomendada** de hacer los Fakes

Es importante que hayamos hecho una buena **inversión de dependencias** para poder aprovechar esta forma!!!

- Podemos crear un objeto de su interface (si la tiene) y **redefinir/implementar** los métodos necesarios



¿Cuándo corro los test unitarios?

- Lo ideal es **correr** los test unitarios **cada vez que cambiamos algo** en el código que estamos probando
 - Eso es **muy seguido...** por eso deben **andar rápido!**
- Claramente, de esta forma, **sólo** vamos a **correr** los **tests vinculados** al código que estamos modificando

Estos grupos de tests son denominados **Compile Suites**, porque básicamente son los tests que vamos a correr cada vez que compilemos

¿Cuándo corro los test unitarios?

- Además de lo anterior, lo recomendable es correr **todos** los tests de unidad algunas veces por día
- El grupo de todos los tests de unidad es denominado **Commit Suite**
 - El nombre no es arbitrario. Es recomendable correr este suite antes de hacer un **commit** en el sistema de versiones
 - Cuanto menos tiempo tarde el Commit Suite, más veces lo vamos a correr en el día

Ejemplo a mano de un Test Unitario

- Retomemos el ejemplo del test unitario que habíamos armado “a mano” para la unidad sumar

```
class Calculador {  
    fun sumar(x: Int, y: Int): Int {  
        return x + y  
    }  
}
```

– El test era:

Arrange

Act

Assert

```
fun main() {  
    val calc = Calculador()  
    val v1 = 2; val v2 = 5; val esperado = 7  
  
    val resultado = calc.sumar(v1, v2)  
  
    if (resultado != 7) {  
        throw IllegalStateException("El resultado de la sumar $v1 con $v2 fue $resultado, pero se esperaba $esperado.")  
    } else {  
        println("Test de sumar($v1, $v2) ejecutado manualmente: ¡Correcto!")  
    }  
}
```

Ejemplo a mano de un Test Unitario

- Retomemos el ejemplo del test unitario que habíamos armado “a mano” para la unidad sumar

```
class Calculador {
    fun sumar(x: Int, y: Int): Int {
        return x + y
    }
}
```

Esta es una correcta **pero muy artesanal** forma de implementar un test unitario

– El test era:

```
fun main() {
    val calc = Calculador()
    val v1 = 2; val v2 = 5; val esperado = 7

    val resultado = calc.sumar(v1, v2)

    if (resultado != 7) {
        throw IllegalStateException("El resultado de la sumar $v1 con $v2 fue $resultado, pero se esperaba $esperado.")
    } else {
        println("Test de sumar($v1, $v2) ejecutado manualmente: ¡Correcto!")
    }
}
```

Arrange

Act

Assert

Usualmente **NO** trabajamos con los test unitarios de esta manera...

Testing

Introducción a JUnit

Contexto

- **JUnit** es un framework de **testing unitario** para Java
- Viene de la familia de herramientas conocida como xUnit que surgió en Smalltalk (SUnit)
 - Cualquier implementación de xUnit está caracterizada por:
 - **Test Runner:** es el ambiente de ejecución de los tests
 - **Test Cases:** la herramienta para programar los tests
 - **Aserciones:** la herramienta utilizada para verificar que el comportamiento o resultado de la unidad testeada es el esperado

Contexto

- **JUnit** es un framework de **testing unitario** para Java

- Viene de la familia de frameworks como xUnit que surgió

Los **IDEs modernos** (IntelliJ, Eclipse, etc.) nos dan la posibilidad de utilizar **JUnit** y por lo tanto **integran** estas **herramientas** al entorno de desarrollo

- Cualquier implementación de JUnit es caracterizada por:

- **Test Runner:** es el ambiente de ejecución de los tests
- **Test Cases:** la herramienta para programar los tests
- **Aserciones:** la herramienta utilizada para verificar que el comportamiento o resultado de la unidad testeada es el esperado

JUnit - ¿En qué nos ayuda?

- Tareas de vinculadas al Testing Unitario

A mano

- Seteo de la Unidad
- Proveer entrada de la Unidad
- Determinar salida esperada
- Ejecutar el Test
- Verificar salida
- Alertar ante el fallo del test

JUnit

- Seteo de la Unidad
- Proveer entrada de la Unidad
- Determinar salida esperada
- ~~• Ejecutar el Test~~
- ~~• Verificar salida~~
- ~~• Alertar ante el fallo del test~~

Esto es lo que
automatiza JUnit

JUnit - ¿En qué nos ayuda?

- Tareas de vinculadas al Testing Unitario

A mano

- Seteo de la Unidad
- Proveer entrada de la Unidad
- Determinar salida esperada
- Ejecutar el Test
- Verificar salida
- Alertar ante el fallo del test

JUnit

- Seteo de la Unidad
- Proveer entrada de la Unidad
- Determinar salida esperada
- ~~Ejecutar el Test~~
- ~~Verificar salida~~

Además, nos da un entorno para poder especificar esto

Creando Tests en JUnit

- En JUnit los tests de unidad se agrupan en clases, llamadas clases de test
- En esas clases los tests son métodos que tienen la anotación `@Test`

Ejemplo (esquemático) de una clase de test para el calculador

```
class Calculador {  
    fun sumar(x: Int, y: Int): Int {  
        return x + y  
    }  
}
```

```
class CalculadorTest {  
    @Test  
    fun `test sumar debe retornar la suma correcta`() {...}  
  
    @Test  
    fun `test restar debe retornar la diferencia correcta`() {...}  
}
```


Creando Tests en JUnit

- En JUnit los tests de unidad se agrupan en clases, llamadas clases de test
- En esas clases los tests son métodos que tienen la anotación `@Test`

Ejemplo (esquemático) de una clase de test para el calculador

```
class Calculador {  
    fun sumar(x: Int, y: Int): Int {
```

Básicamente **correr estas funciones** implicará **ejecutar los tests**. JUnit los invoca **automáticamente**

```
class CalculadorTest {  
    @Test  
    fun `test sumar debe retornar la suma correcta`() {...}  
  
    @Test  
    fun `test restar debe retornar la diferencia correcta`() {...}  
}
```

Creando Tests en JUnit

- En una clase de test también podemos tener **otro tipo de funciones** que se comportaran según su **anotación**
- Por ejemplo, los más comunes son:
 - **@BeforeAll**: El método (debe ser estático!) se va a ejecutar una vez **antes de ejecutar todos los tests**
 - **@BeforeEach**: El método se va a ejecutar **antes de ejecutar cada test**
 - **@AfterAll**: El método (debe ser estático!) se va a ejecutar una vez **terminada la ejecución de todos los tests**
 - **@AfterEach**: El método se va a ejecutar cada vez que **termina la ejecución de cada test**

Creando Tests en JUnit

- En una clase de test también podemos tener **otro tipo de funciones** que se comportaran según su **anotación**
- Por ejemplo, los más comunes son:
 - **@BeforeAll**: El método (de) ejecutar una vez **antes de**
 - **@BeforeEach**: El método ejecutar **cada test**
 - **@AfterAll**: El método (de) ejecutar una vez **terminada la ejecución de todos los tests**
 - **@AfterEach**: El método se va a ejecutar cada vez que **termina la ejecución de cada test**

Estos métodos en general nos sirven para el **Arrange** y para limpieza del ambiente común a los tests, de forma que nos evitemos el código repetido dentro de los tests

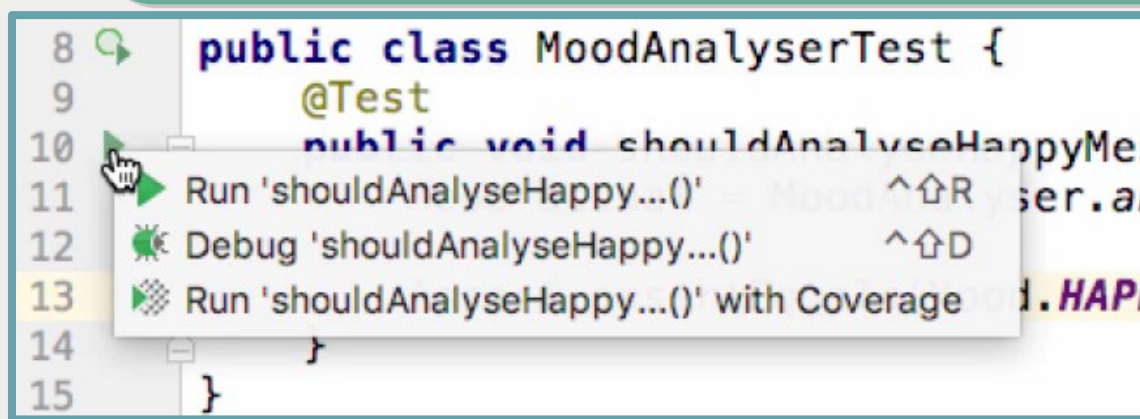
Creando Tests en JUnit – Prácticas comunes

- La práctica usual es tener al menos **una clase de test por clase/modulo** que queremos probar
 - Los IDEs nos dan herramientas para **automatizar la creación** de una clase de test esqueleto a partir de métodos y clases de nuestro código base
 - En general, la **clase de test** la ubicamos en un paquete de “tests” dentro del **mismo paquete** de la **clase** que estamos testeando
 - Los IDEs suelen seguir estas políticas por defecto

Ejecución de los Tests

- Para ejecutar tests la plataforma de JUnit toma la clase de test asociada, crea **un objeto por cada test** e invoca al **método** correspondiente (además de todos los métodos `beforeEach` y `afterEach` correspondientes)

Los IDEs nos dan herramientas para correr los tests de una clase de test directamente desde el ambiente de desarrollo (apuntando a clase y eligiendo correr, por ejemplo)

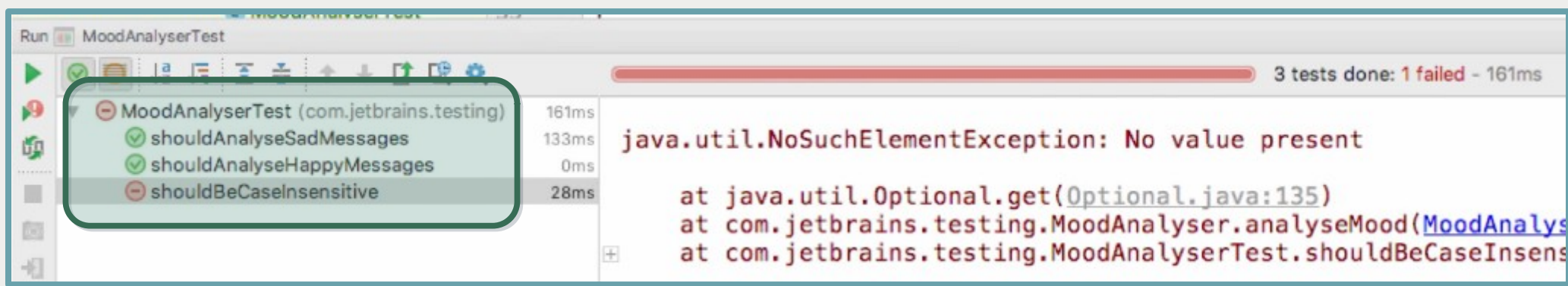


screenshot de IntelliJ

Ejecución de los Tests

- La plataforma de JUnit, para cada test que corre, indica si tiene Éxito o Falla

Los IDEs integran herramientas para visualizar este proceso indicándonos los test que tuvieron éxito en verde y los que fallaron en rojo

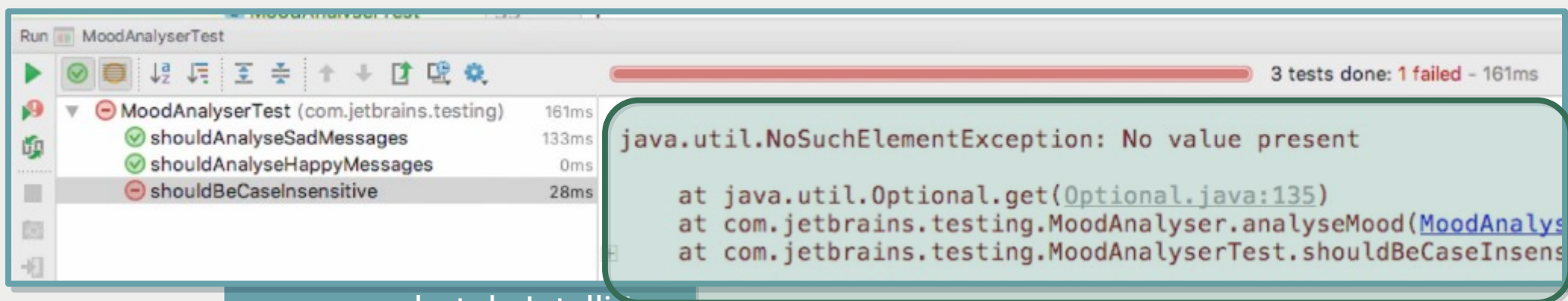


screenshot de IntelliJ

Ejecución de los Tests

- La plataforma de JUnit, para cada test que corre, indica si tiene Éxito o Falla

Los IDEs también nos muestran las razones de los tests fallidos



screenshot de IntelliJ

Ejecución de los Tests

- Para determinar si un test tiene éxito o falla JUnit tiene la siguiente política:
 - Si el **método** de test **termina su ejecución** correctamente, el test tiene **Éxito**
 - Si el **método de test termina** su ejecución de manera **anormal** (por ejemplo, porque propaga/lanza una excepción) el test **Falla**

Código de los Tests

```
class CalculadorTest {  
    @Test  
    fun `test sumar debe retornar la suma correcta`() {...}  
  
    @Test  
    fun `test restar debe retornar la diferencia corre  
}
```



¿Qué va adentro de los métodos `@Test`?

Ese es el cuerpo del test, así que ahí vas a poner código

¿Cualquier código?

SI! siempre y cuando compile... Sin embargo, la idea es poner sólo el código que caracterice al test. JUnit nos da primitivas útiles para esa caracterización

Código de los Tests

- El código de los métodos de tests debería seguir la forma de un test de unidad:
 - **Arrange**: Seteo del ambiente y datos del test
 - **Act**: Invocación de la unidad
 - **Assert**: que la salida es la esperada

Código de los Tests

- El código de los métodos de tests debería seguir la forma de un test de unidad:
 - **Arrange**: Seteo del ambiente y datos del test
 - **Act**: Invocación de la unidad
 - **Assert**: que la salida es la esperada

```
class CalculadorTest {  
    @Test  
    fun `test sumar debe retonar la suma correcta`() {  
        //Arrange  
        val calc = Calculador()  
        val x = 2; val y = 5; val esperado = 7  
        //Act  
        val resultadoObtenido = calc.sumar(x, y)  
        // Assert  
        assertEquals(esperado, resultadoObtenido)  
    }  
}
```

Código de los Tests

- El código de los métodos de tests debería seguir la forma de un test de unidad:

- **Arrange**: Seteo del ambiente y datos del test
- **Act**: Invocación de la unidad
- **Assert**: que la salida

Puede involucrar crear objetos y setear dependencias. Algunas actividades pueden hacerse con los métodos `@BeforeEach`

```
class CalculadorTest
    @Test
    fun `test sumar`() {
        //Arrange
        val calc = Calculador()
        val x = 2; val y = 5; val esperado = 7
        //Act
        val resultadoObtenido = calc.sumar(x, y)
        // Assert
        assertEquals(esperado, resultadoObtenido)
    }
}
```

Código de los Tests

- El código de los métodos de tests debería seguir la forma de un test de unidad:
 - **Arrange**: Seteo del ambiente y datos de entrada
 - **Act**: Invocación de la unidad
 - **Assert**: que la salida es la esperada

A veces los datos de entrada se determinan directamente en la llamada

```
class CalculadorTest {  
    @Test  
    fun `test sumar debe retonar la suma correcta`() {  
        //Arrange  
        val calc = Calculador()  
        val x = 2; val y = 5; val esperado = 7  
        //Act  
        val resultadoObtenido = calc.sumar(x, y)  
        // Assert  
        assertEquals(esperado, resultadoObtenido)  
    }  
}
```

Código de los Tests

- El código de los métodos de tests debería seguir la forma de un test de unidad
 - **Arrange**: Seteo del ambiente
 - **Act**: Invocación de la unidad
 - **Assert**: que la salida es la esperada

Debemos comparar el resultado obtenido con el esperado y ver si son compatibles

```
class CalculadorTest {  
    @Test  
    fun `test sumar debe retonar la suma correcta`() {  
        //Arrange  
        val calc = Calculador()  
        val x = 2; val y = 5; val esperado = 7  
        //Act  
        val resultadoObtenido = calc.sumar(x, y)  
        // Assert  
        assertEquals(esperado, resultadoObtenido)  
    }  
}
```

Código de los Tests

- El código de los métodos de tests debería seguir la forma de un test de unidad:
 - **Arrange**: Seteo del ambiente y datos del test
 - **Act**: Invocación de la u
 - **Assert**: que la salida es

Lo que hace este **assert** es ver si el segundo parámetro es igual (equals de java) al primero; si no lo son lanza una excepción cuyo mensaje indica la discrepancia

```
class CalculadorTest {  
    @Test  
    fun `test sumar debe retornar`() {  
        //Arrange  
        val calc = Calculador()  
        val x = 2; val y = 5; val esperado = 7  
        //Act  
        val resultadoObtenido = calc.sumar(x, y)  
        // Assert  
        assertEquals(esperado, resultadoObtenido)  
    }  
}
```

Testing

Test Fakes en JUnit

La realidad es que para los Fakes no tenemos herramientas especiales en Junit asi que directo veamos un ejemplo

Definición y Conceptos – Test Fakes

Veamos un test unitario en JUnit usando un fake para el ejemplo

```
class Pedido(  
    private val cliente: ICliente,  
    private val producto: IProducto,  
    private val cantidadProducto: Int  
) {  
    fun calcularPrecioPedido(): Int {  
        val valorNominal = producto.precio() * cantidadProducto  
        val descuento = valorNominal * cliente.descuentoPorcentaje()  
        return (valorNominal - descuento).toInt()  
    }  
}
```

Definición y Conceptos – Test Fakes

Veamos un test unitario en JUnit usando un fake para el ejemplo

```
class Pedido(  
    private val cliente: ICliente,  
    private val producto: IProducto,  
    private val cantidadProducto: Int  
) {  
    fun calcularPrecioPedido(): Int {  
        val valorNominal = producto.precio() * cantidadProducto  
        val descuento = valorNominal * cliente.descuentoPorcentaje()  
        return (valorNominal - descuento).toInt()  
    }  
}
```

Método a
testear

Definición y Conceptos – Test Fakes

Veamos un test unitario en JUnit usando un fake para el ejemplo

```
class Pedido(  
    private val cliente: ICliente,  
    private val producto: IProducto,  
    private val cantidadProducto: Int  
) {  
    fun calcularPrecioPedido(): Int {  
        val valorNominal = producto.precio() * cantidadProducto  
        val descuento = valorNominal * cliente.descuentoPorcentaje()  
        return (valorNominal - descuento).toInt()  
    }  
}
```

Fake de
Cliente

```
class ClienteFake : ICliente {  
    override fun descuentoPorcentaje(): Float {  
        return 0.1f // Descuento fijo del 10%  
    }  
}
```

Definición y Conceptos – Test Fakes

Veamos un test unitario en JUnit usando un fake para el ejemplo

```
class Pedido{
    private val cliente: ICliente,
    private val producto: IProducto,
    private val cantidadProducto: Int
} {
    fun calcularPrecioPedido(): Int {
        val valorNominal = producto.precio() * cantidadProducto
        val descuento = valorNominal * cliente.descuentoPorcentaje()
        return (valorNominal - descuento).toInt()
    }
}
```

Test unitario para
calcularPrecioPedido

```
class ClienteFake : ICliente {
    override fun descuentoPorcentaje(): Float {
        return 0.1f // Descuento fijo del 10%
    }
}
```

```
@Test
fun `calcularPrecioPedido con fake de cliente`() { //
    // Arrange
    val cliente: ICliente = ClienteFake() // Usando el fake
    val producto: IProducto = Producto( nombre: "compu", precioBase: 100)
    val pedido = Pedido(cliente, producto, cantidadProducto: 5)
    // Act
    val resultadoObtenido = pedido.calcularPrecioPedido()
    // Assert
    val resultadoEsperado = 450 // (100 * 5) - (500 * 0.1) = 500 - 50 = 450
    assertEquals(resultadoEsperado, resultadoObtenido)
```